

Infrastructure as Documentation:

Designing IaC That Explains Itself

Chad Green

Senior Systems Architect
Jasper Engines & Transmissions

September 29, 2026
10:30 – 11:15 AM EDT





The Consequences

Fear to Change

New engineers were afraid to touch anything

Blind Reviews

PR reviews became rubber stamps; no one knew what they were approving

Governance Drift

Governance drifted into a Confluence page nobody read

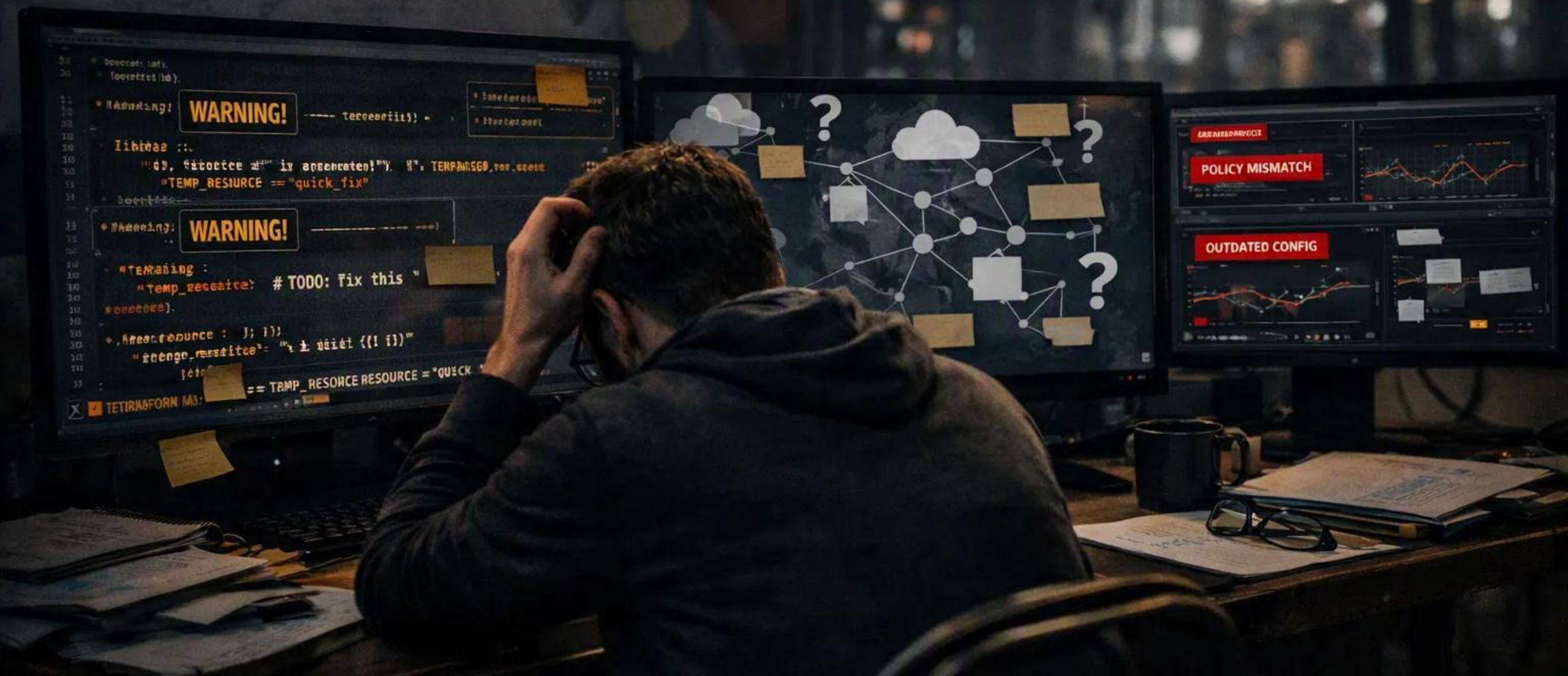
Haunted Deploys

Every deployment felt like spelunking in a haunted forest

Terraform wasn't the problem. *The intent had disappeared.*

The Problem: IaC Rot and Invisible Intent

Three failure modes. Every team has them.



What IaC Rots



Failure Mode 1

Invisible Intent

Code shows **what it builds** — never **why**. Names, variables, and outputs carry zero explanation of purpose or decision rationale.

🚫 The *why* behind every resource is invisible to future engineers.

What IaC Rots



Failure Mode 1

Invisible Intent

Code shows **what it builds** — never **why**. Names, variables, and outputs carry zero explanation of purpose or decision rationale.

🗑️ The *why* behind every resource is invisible to future engineers.



Failure Mode 2

Governance Drift

Standards documents **diverge from the infrastructure they govern**. Policy lives in Confluence; the code never gets the memo.

🗑️ The PDF was perfect. The infrastructure was not.

What IaC Rots



Failure Mode 1

Invisible Intent

Code shows **what it builds** — never **why**. Names, variables, and outputs carry zero explanation of purpose or decision rationale.

🚫 The *why* behind every resource is invisible to future engineers.



Failure Mode 2

Governance Drift

Standards documents **diverge from the infrastructure they govern**. Policy lives in Confluence; the code never gets the memo.

📄✗ The PDF was perfect. The infrastructure was not.



Failure Mode 3

Review Fatigue

PRs become **rubber-stamps** when context is missing. Reviewers can't catch what they can't understand — so they approve and hope.

👤 LGTM becomes the most dangerous phrase in infrastructure.

What IaC Rots



Failure Mode 1

Invisible Intent

Code shows **what it builds**;
never **why**. Names, variables,
and outputs carry zero
explanation of purpose or
decision rationale.

 The *why* behind every resource is
invisible to future engineers.



Failure Mode 2

Governance Drift

Standards documents
**diverge from the
infrastructure they govern.**
Policy lives in Confluence; the
code never gets the memo.

 The PDF was perfect. The
infrastructure was not.



Failure Mode 3

Review Fatigue

PRs become **rubber stamps**
when context is missing.
Reviewers can't catch what they
can't understand, so they
approve and hope.

 LGTM becomes the most
dangerous phrase in infrastructure.



Show of hands: How many of you inherited Terraform you didn't write and didn't immediately understand?

Aurora's Reality



Aurora's Reality

</> The Code

```
main_vnet
```

```
// wasn't the main VNet
```

```
env
```

```
// no description, no validation
```

```
all_subnet_ids
```

```
// a flat list of implementation details
```

⚠ The Cost

- New engineers can't orient themselves
- Reviews can't catch what reviewers don't understand

Aurora's Reality

</> The Code

```
main_vnet
```

```
// wasn't the main VNet
```

```
env
```

```
// no description, no validation
```

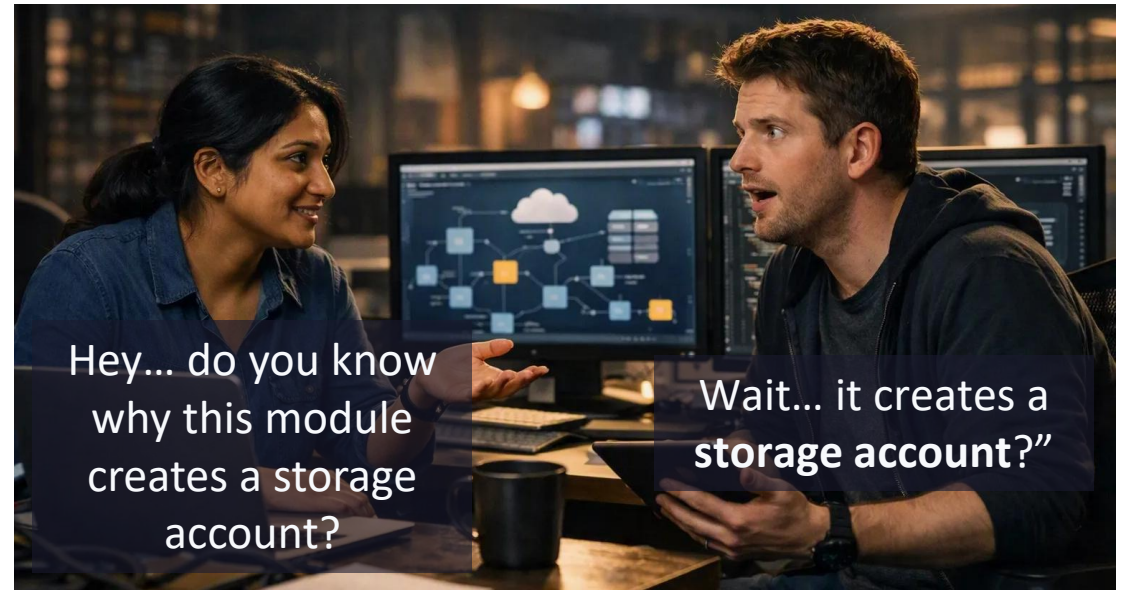
```
all_subnet_ids
```

```
// a flat list of implementation details
```

```
resource azurerm_storage_account "example" {  
  name                = "storageaccountname"  
  resource_group_name = azurerm_resource_group.this.name  
  location             = azurerm_resource_group.this.location  
  account_tier         = "Standard"  
  account_replication_type = "GRS"  
}
```

⚠ The Cost

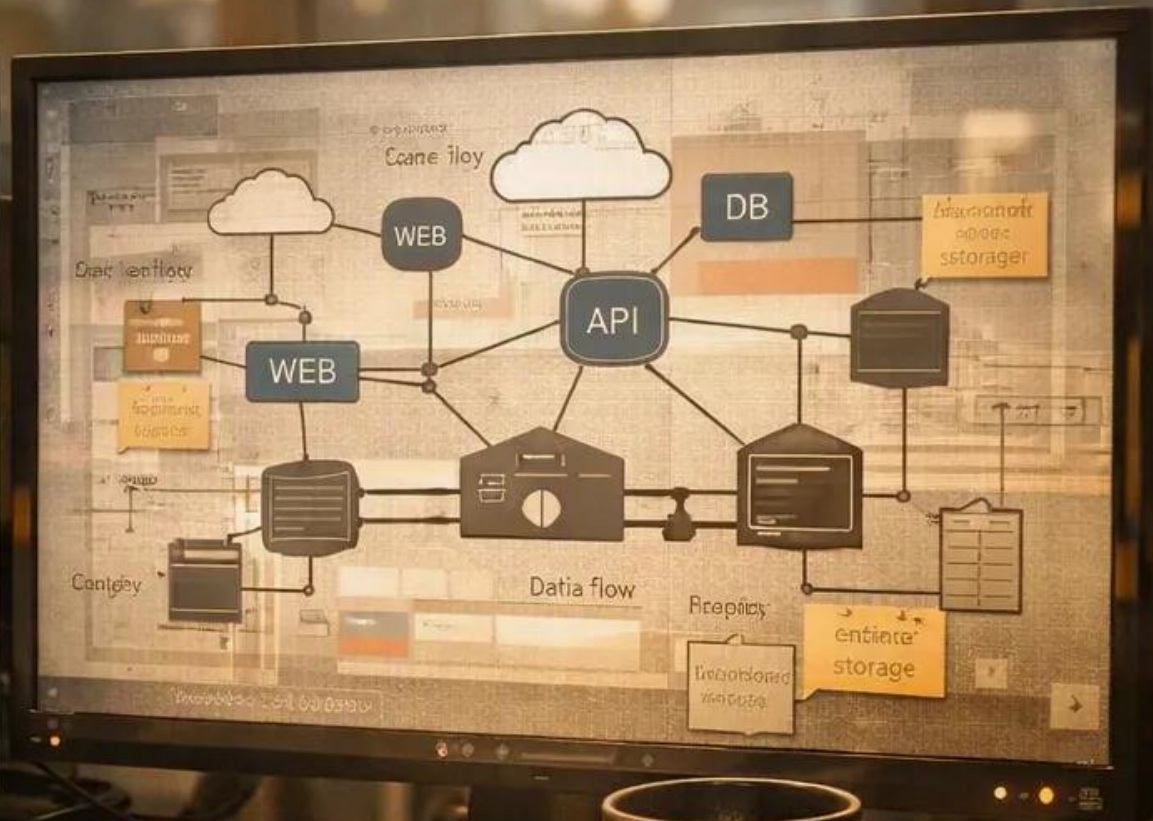
- New engineers can't orient themselves
- Reviews can't catch what reviewers don't understand



The Reframe: Infrastructure as Documentation

A different lens on what IaC is fundamentally for.

```
<module "vpc" app_network: {  
  description = "VPC for application:  
  cidr_block = "10.0.0.0/16"  
  
  <variable "project_env" {    → Project Environment,  
    description = "Project Environment"  
    type = "string" = "production"  
  
  <output "vpc_id" {  
    < description = "VPC IP"    → Value=module.vpc.id  
  < // Docs: Security group 'web_sg' allows HTTP& HTTPS traffic  
  
  resource "aws_security_group" web_sg {  
    < { : name = "Web SG"  
    < { : default = "protocol=*" < ingress
```



LSTN

"IaC is not automation — it's a communication artifact."

Written for people to read, reason about, review, and build on; that also happens to be executable.

“Modules are not scripts — they are contracts.”

A module defines an agreement between the producer and every future consumer: what it accepts, what it guarantees, and what it will never do implicitly.

“Variables and outputs are documentation surfaces.”

This is the first thing a new engineer reads when they open a module. Every name, description, default, and validation rule is a communication decision.

“Naming is architecture in language.”

Every resource name, every module name, every variable name is an architectural decision made visible (or invisible) by how deliberately you choose your words. The name is not decoration. The name *is* the documentation.

Infrastructure as Documentation

“IaC is not automation — it’s a communication artifact.”

“Modules are not scripts — they are contracts.”

“Variables and outputs are documentation surfaces.”

“Naming is architecture in language.”



The Contract-First Mindset

Four pillars for designing infrastructure that communicates intent.



Intent

What is this module for? One sentence before the first resource block.



Boundaries

Modules map to architectural boundaries, not convenience groupings.



Assumptions

Every assumption must be visible, not buried in defaults.



Contracts

Outputs declare stable, named, described interfaces; not implementation leaks.

The Contract-First Mindset

Four pillars for designing infrastructure that communicates intent.



Intent



Boundaries



Assumptions



Contracts



“Each pillar eliminates a category of invisible failure.”

The Pattern Library: How Code Explains Itself

Five patterns. Portable.

Grounded in contract-first thinking.

Intent-First Naming

```
bucket_name = "audit_logs"
```

Audit Logs

Assumption Surfacing

```
default_region = "us-east-1"
```

Default Setting

Outputs as Contracts

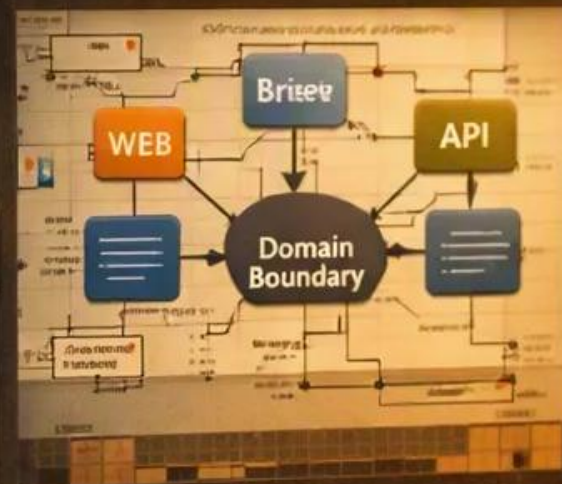
```
output "db_host" {  
  value = db.endpoint
```

Stable Interface

Opinionated Defaults

```
encryption_enabled = true
```

Team Standard:
Encryption On



Pattern Library Overview

Five portable patterns, each named, explained, and tied to contract-first principles. Works across every IaC tool.

1

Intent-First Naming

Names describe purpose, not implementation



2

Assumption Surfacing

Defaults encode visible, enforceable decisions



3

Boundary-Driven Module Structure

Modules map to architectural domains, not convenience groupings



4

Outputs as Contracts

Expose stable interfaces, not implementation details



5

Opinionated Defaults

Encode your team's standards, not the provider's factory settings



Pattern 1: Intent-First Naming

⚠ Naming That Lies

main_vnet

Was not the main VNet

db1

Which database? For which service?

sg-main

The main SG for what?

env

What values are valid?

✗ The Damage

- ✗ Communicates wrong intent — worse than no name at all
- ✗ New engineers guess; guesses calcify into accidental architecture
- ✗ Bad naming is a lie encoded in infrastructure



Pattern 1: Intent-First Naming

✗ BEFORE — What It Is

db1

Which database? For which service?

sg-main

The main SG for... what exactly?

env

What values are valid? Any validation?

main_vet

Wasn't the main VNet — it was staging

→
Rename
for
intent

✓ AFTER — What It's For

payments_postgres_primary

Payments service, Postgres, primary node

internal_api_allow_https

Internal API traffic, HTTPS only

deployment_environment

With validation block; enforced at plan time

private_app_network

Private network for application workloads

💡 *The name should answer "what is this for" — not "what is this."*

Pattern 2: Assumption Surfacing



"As Aurora dug deeper, they found landmines everywhere. Hidden assumptions don't disappear. They wait."

Pattern 2: Assumption Surfacing

Hidden Landmines

- ⚠️ **Region hardcoded to `eastus`**
No one made that decision deliberately; it was never revisited.
- ⚠️ **TLS version defaulted to `1.0`**
Set when the module was first written; never updated after the security policy changed.
- ⚠️ **Public IPs allowed on every VM**
The security memo arrived. The IaC didn't get it.



Hidden Assumptions

- **Region: `eastus` (hardcoded)**
- **TLS `1.0` (legacy default)**
- **Public IPs: allowed (default)**

*Hidden assumptions don't disappear.
They wait.*

"As Aurora dug deeper, they found landmines everywhere. Hidden assumptions don't disappear. They wait."

Pattern 2: Assumption Surfacing

Technique 1

Variable Validation

Explicitly allow values and error messages at **plan time** — not at 2 AM during an incident.

Technique 2

Explicit Defaults with Descriptions

`default`: "TLS1_2" with a description that cites the security policy date — the why is baked in.

Technique 3

Purposeful Absence of Defaults

No default = a visible decision point. Forces a conscious choice every deployment; nothing hides.



Surfaced assumptions are governance wins; every visible constraint is a policy enforced at plan time.

Terraform — Surfaced Assumptions

```
variable "deployment_environment" {
  type = string
  # No default — forces conscious choice
  description = "Target environment (prod/staging/dev)"
  validation {
    condition = contains(
      ["prod","staging","dev"] , var.deployment_environment)
    error_message = "Must be prod, staging, or dev."
  }
}

variable "tls_version" {
  type = string
  default = "TLS1_2" # Per policy 2024-03
  description = "Min TLS version. See security
policy updated 2024-03-01."
  validation {
    condition = contains(
      ["TLS1_2","TLS1_3"] , var.tls_version)
    error_message = "TLS 1.0/1.1 not permitted."
  }
}
```

Pattern 3: Boundary-Driven Module Structure

- Aurora’s “networking” module created: VNets, subnets, NSGs, route tables — and a random storage account
- The storage account was added for flow logs. The requirement went away. The storage account stayed.
- Nobody noticed; the file was already too big to read carefully.



BLOB MODULE

networking/ module

```
🌐 azurerm_virtual_network
🌐 azurerm_subnet
🛡️ azurerm_network_security_group
🌐 azurerm_route_table
📁 azurerm_storage_account ← shouldn't be here
```

“ *This is what happens when modules are organized by convenience instead of architectural boundaries.* ”

Pattern 3: One Module, One Boundary

network/

VNets, subnets, NSGs, peerings. Network concerns only. Full stop.

compute/

Virtual machines, scale sets, container instances. Compute concerns only.

security/

Identity, certificates, key vaults, security policies. One concern. Clear ownership.

"When each module maps to one architectural boundary: dependencies become visible, ownership becomes clear, reviews become meaningful."

```
modules/
├─ network/
│  ├─ main.tf
│  ├─ variables.tf
│  └─ outputs.tf
├─ compute/
│  ├─ main.tf
│  └─ variables.tf
└─ security/
   ├─ main.tf
   └─ outputs.tf
one folder · one concern · clear ownership
```

Pattern 4: Outputs as Contracts – The Leaky Version

- Output: `all_subnet_ids` is a flat list of every subnet ID in the module, **in order of creation**
- Three downstream modules accessed subnets by index position `[0]` app tier, `[1]` data tier, `[2]` management
- Team added a DMZ subnet → **indices shifted** → three modules broke silently at deployment, **Friday at 11 PM**



LEAKY OUTPUT

BEFORE DMZ SUBNET

`[0]` app tier
`[1]` data tier
`[2]` management

AFTER DMZ ADDED

`[0]` dmz ← shifted
`[1]` app tier ← shifted
`[2]` data tier ← shifted
`[3]` management ← shifted

3 modules broke silently

⚠ "That is not an output. That is a leak."

Patterns 4 & 5: Contracts + Opinionated Defaults

⚠ Leaky Output

`all_subnet_ids`

Undifferentiated flat list — consumers depend on index

positions: `[0]` app tier

`[1]` data tier

`[2]` management

Add one subnet → indices shift → three modules break silently at 11 PM on a Friday.

✓ Contract-Driven Output

`app_tier_subnet_id`

`data_tier_subnet_id`

`management_subnet_id`

Singular, named, architectural role. Stable interface; adding subnets does not break consumers.

Patterns 4 & 5: Contracts + Opinionated Defaults

⚠ Leaky Output

`all_subnet_ids`

Undifferentiated flat list — consumers depend on index

positions: `[0]` app tier

`[1]` data tier

`[2]` management

Add one subnet → indices shift → three modules break silently at 11 PM on a Friday.

✓ Contract-Driven Output

`app_tier_subnet_id`

`data_tier_subnet_id`

`management_subnet_id`

Singular, named, architectural role. Stable interface; adding subnets does not break consumers.

Pattern 5: Opinionated Defaults



Private endpoint enabled by default

Not public access



Deletion protection on by default



Minimum TLS version enforced

What your security policy requires — not the provider's factory setting

"The default is the policy."

Live Teardown: Fixing a Messy Terraform Module

Aurora's actual module.
Step by step.
Narrated in real time.

```
1 # Define VPC with clear intent
2 module 'core_vpc' {
3   purpose = "app-network"
4   cidr_block = var.app_cidr
5 }
6
7 # Surface Assumptions
8 variable 'db_instance_count' {
9   description = "Number of DB instances (Min 2)"
10  default = 2
11 }
12
13 ## Output Prod Endpoint
14 output "prod_endpoint" {
15   value = module.web.endpoint
16 }
```

Intent-first
naming ✓

- Intent-first naming
- Assumption Surfacing
- Boundary-Driven Modules
- Outputs as Contracts
- Opinionated Defaults

Aurora's Module: The Before

```
1 # Aurora's networking module – networking.tf
2
3 variable "env" {
4   type = string # no description, no validation
5 }
6
7 variable "region" {
8   default = "eastus" # hardcoded – no rationale
9 }
10
11 resource "azurerm_virtual_network" "main_vnet" {
12   name = "main-vnet-${var.env}"
13   location = var.region
14 }
15
16 resource "azurerm_network_security_group" "sg1" {
17   name = "sg1-${var.env}"
18   location = var.region
19 }
20
21 resource "azurerm_network_security_group" "sg2" { ... }
22 resource "azurerm_route_table" "rt_default" { ... }
23 resource "azurerm_public_ip" "pip_001" { ... }
24 resource "azurerm_storage_account" "logs" { ... }
25
26 output "resource_ids" {
27   value = [ azurerm_virtual_network.main_vnet.id ,
28     azurerm_network_security_group.sg1.id , ... ] # index-dep
29 }
```

⚠ NAMING CRIMES

main_vnet, sg1, sg2, pip_001; what are they for?

⚠ BLOB STRUCTURE

VNets, NSGs, storage, public Ips; mixed concerns

⚠ HIDDEN ASSUMPTIONS

region hardcoded to “eastus”; no rationale visible

⚠ LEAKY OUTPUT

resource_ids; index-dependent list, no description

Teardown: Six Steps to Clarity

1 Identify the Intent

One sentence for what this module is actually for — before the first resource block.

2 Extract Boundaries

Move VMs to `compute/`, key vault to `security/`, storage to `observability/`. One concern per module.

3 Rename for Intent

`main_vnet` → `private_app_network` · `sg1 app_tier_allow_https`

4 Surface Assumptions

Validation blocks, explicit defaults, descriptions for every variable. No hidden decisions.

5 Rebuild Outputs as Contracts

Remove `all_resource_ids`. Replace with named, described outputs that declare a stable architectural interface.

6 Show the Diff

Same infrastructure. Different communication. Left: holds intent hostage. Right: teaches.

Aurora's Module: The After

```
# module: private-app-network
# Purpose: Provisions isolated VNet topology for the private application tier.
# Scope: networking only – no compute, no identity, no observability.

variable "deployment_environment" {
  description = "Target env. Governs naming, tagging, peering topology."
  validation {
    condition = contains(["prod","staging","dev"], var.deployment_environment)
    error_message = "Must be prod, staging, or dev."
  }
}

variable "minimum_tls_version" {
  description = "Min TLS. Default enforces Security Policy SP-2023-04."
  default = "TLS1_2"
}

resource "azurerm_virtual_network" "private_app_network" { ... }
resource "azurerm_network_security_group" "app_tier_allow_https" { ... }
resource "azurerm_network_security_group" "data_tier_deny_public" { ... }

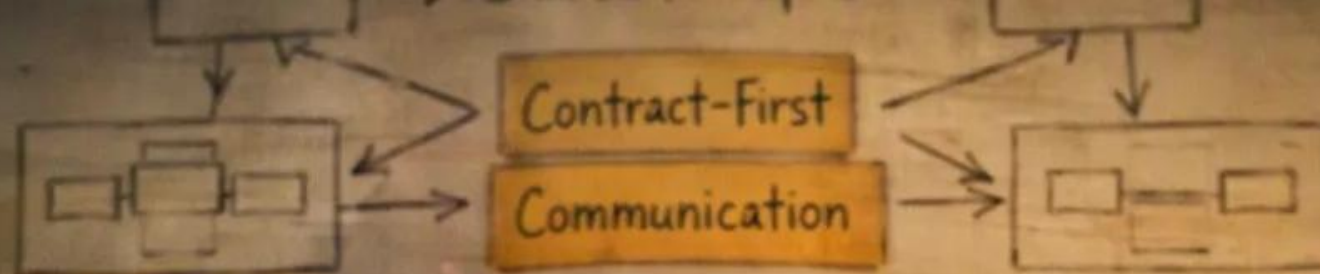
output "app_tier_subnet_id" { description = "App-tier subnet. Stable contract." }
output "data_tier_subnet_id" { description = "Data-tier subnet. Isolated from public ingress." }
output "management_subnet_id" { description = "Ops/management plane subnet." }
```

- ✓ INTENT-FIRST NAMES
- ✓ CLEAN BOUNDARIES
- ✓ SURFACED ASSUMPTIONS
- ✓ CONTRACT OUTPUTS

Payoffs

- Reviews take half the time
- Onboarding accelerates
- Governance becomes durable





Cross-Tool Translation: The Patterns Are Universal

Terraform is the medium. The principles are the message.



The Discipline Travels. The Tool Doesn't Have To.

Terraform

- Named outputs with `description` fields
- Variable validation blocks with explicit allowed values
- Module folders mapped per architectural domain

Bicep

- `@description()` decorator on parameters, outputs, resources
- `@allowed()` constraints for visible decisions
- Separate `.bicep` files per architectural boundary

Pulumi

- JSDoc on `ComponentResource` classes
- TypeScript interfaces as explicit contracts
- Separate stacks per architectural boundary

AWS CDK

- L3 Constructs as opinionated, boundary-scoped modules
- README-style docstrings on Construct classes
- `CfnOutput` with description fields for every interface

“Aurora's tool debates changed: from 'Terraform versus Pulumi' to 'Is this module's intent visible?'”

Aurora's Realization

**The tool does not matter.
The discipline does.**

Whether you are writing Terraform HCL or TypeScript or Python or YAML
— the questions are the same: Does this module communicate its
intent? Do its outputs declare a contract? Are its assumptions visible?

Governance Without Bureaucracy

Standards that live in the code; where they can actually be enforced.

```
1 # Enforce Encryption
```

```
2 resource "aws_s3_bucket" "secure_bucket" {  
3   encryption = "AES256"  
4 }  
5
```

```
6 # Require Tagging
```

```
7 tags = {  
8   Environment = "prod"  
9 }  
10 Owner = var.owner
```

```
11 # Validate Region
```

```
12 assert(var.region == "us-east-1", "Region must be us-east-1")  
13  
14 }
```

Governance Rules

- ✓ Encryption Enabled
- ✓ Required Tags Present
- ✓ Region Validated
- ✓ Policy Passed!

The PDF Nobody Read

- **Cloud Infrastructure Governance Standards, Version 3.2**; 14 pages; distributed via Confluence link in a filtered email
- When engineers violated a standard: an email pointing to **page 11 of the PDF**
- Standards drifted · security rules misconfigured · TLS crept back to insecure defaults · region pinning forgotten during **“quick” test deployments**

"The PDF was perfect. The infrastructure was not. And the gap between them grew wider with every deployment."

Governance documents drift because they live outside the system they govern.



⚠ Last opened: unknown

📄 14 pages · Confluence-only distribution

✉ Enforcement via email → page 11

Governance in Code: Three Self-Enforcing Mechanisms



Mechanism 1

Opinionated Defaults as Policy

Every deployment automatically applies your security posture. Overrides require explicit, visible code in the PR diff — making exceptions intentional and auditable, never accidental.



Mechanism 2

Validation Blocks as Policy Enforcement

Engineers who violate policy get immediate plan-time errors — not post-audit findings six weeks later. Policy violations surface at the earliest possible moment in the delivery pipeline.



Mechanism 3

Outputs as Audit Trail

Named, described outputs create a machine-readable architecture map. Automated tooling can answer governance questions continuously — not just at audit time.

Governance that is self-enforcing because it is in the code.

A man and a woman are sitting on a stone path in a lush garden at sunset. The woman is holding a tablet and the man is holding a notebook. They are both smiling and looking at each other. The garden is filled with various flowers, including sunflowers and purple flowers. In the background, there is a small pond with a fountain and a house with a porch. The scene is bathed in the warm, golden light of the setting sun.

The haunted forest became a garden.

Every module told a story. Every variable revealed a decision.

Five Things to Take Home

1 IaC is Documentation

Write infrastructure to be **read, not just executed** — the reader is always more important than the API.

2 Modules Are Contracts

Design **interfaces, not scripts**. What you accept and produce defines boundaries and promises.

3 Intent Must Be Visible

If the **why is invisible**, the code is incomplete. Name things for purpose, not implementation.

4 Architecture Is Encoded in Code

Folder structure, module boundaries, naming conventions — **deliberate architectural decisions**.

5 Clarity Is a Feature

Not a luxury. Not a nice-to-have. Your future self, your new hires, and your next Tuesday morning all depend on it.

“ These are not Terraform best practices. These are **communication principles**. They will outlast every tool you are using today.

Thank You

Chad Green

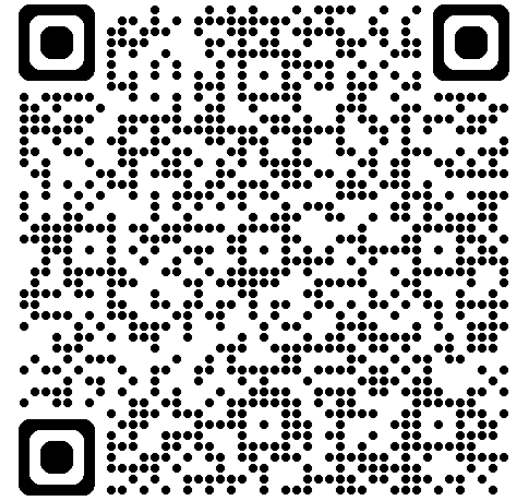
Senior Systems Architect
Jasper Engines & Transmissions

✉ chadgreen@chadgreen.com

🌐 chadgreen.com

🐦 @chadgreen

📘 ChadwickEGreen



Portable Pattern Card + Slides

DevOpsCon New York

September 29, 2026 · 10:30 AM EDT

"If any part of Aurora's story sounded familiar — come find me."